

Native Qwen3-Omni Waveform Runtime and Bounded Audio Context on a 24 GB L4

Implementation Proposal Outline

Systems Architecture and Experimental Plan

August 23, 2026

Implementation sequence at a glance

Stage	Input representation	Historical policy	Purpose and runtime status
A	Full native AuT sequence	Protected prefix plus recent window	Establish the uncompressed, fixed-budget C++/GGUF baseline.
B	Fixed-rate AuT temporal pooling	Protected prefix plus recent window	Establish a deterministic, ASR-free compression baseline at matched token budgets.
C	F16 Whisper-Turbo semantic anchoring	Protected prefix plus recent window	Reproduce VoxZip Stage 1 method on the A100: transcript timestamps, Qwen text embeddings, and aligned AuT pooling.
D	F16 Whisper-Turbo semantic anchoring	VoxZip temporally decayed attention eviction	Reproduce the complete VoxZip method on the A100, documenting any backbone-precision or context-length deviation forced by single-GPU capacity.
E	Fixed-rate AuT temporal pooling	The same temporal eviction used in D	Isolate the value of text-derived semantic anchors by removing Whisper while holding eviction constant.
F	Learned monotonic AuT compactor	Protected prefix plus recent window	Learn transcript-free, chronologically aligned AuT anchors using A100 teacher–student distillation.
G	Learned monotonic AuT compactor	The same temporal eviction used in D and E	Evaluate the intended ASR-free compressed-context method at identical KV bytes.

Whisper precision and residency ladder.

1. Use F16 Whisper-Turbo on the A100 for the unquantized-ASR, method-faithful VoxZip reference in Stages C and D.
2. For the native L4 adaptation, first test Q8_0 Whisper-Turbo. Q8 is preferred because the additional weight cost over Q5_0 is only approximately 0.3 GiB and its numerical perturbation is smaller.
3. Attempt concurrent Q8 residency during listening. Run Whisper in bounded chunks on a separate, lower-priority CUDA stream while AuT and Thinker work remains higher priority.
4. If concurrent residency or compute contention is unacceptable, use sequential Q8 residency: finish AuT, release its weights and workspace, load Whisper into the same arena, construct the anchors, then release Whisper before output generation.
5. Use Q5_0 only if Q8 still exceeds the measured deployment envelope. Apply the same concurrent-then-sequential residency tests to Q5.

6. Treat the F16 A100 result as the unquantized-ASR, method-faithful reference and the Q8/Q5 L4 results as quantized deployment adaptations of the same algorithm.

Project objective

- Execute **Qwen3-Omni-30B-A3B-Instruct** as a native waveform-to-waveform system in one C++ process using GGML and GGUF.
- Fit the complete runtime on one **24 GB NVIDIA L4**; this is a hard deployment constraint.
- Preserve the intelligence-critical Thinker at **Q4_K_M** and the Talker at **Q8_0**.
- Bound the serving process’s live neural state, queues, runtime ledgers, and retained audio buffers independently of interaction length. Append-only audit evidence may grow on disk after the serving process releases the serialized records and source buffers.
- Avoid Python serving and external TTS. Whisper is admitted only as a VoxZip reference baseline; the intended final method remains ASR-free.
- Expose sufficient native runtime state to instrument allocation, routing, attention, and KV behavior and to implement custom bounded-memory policies.

1 Native Qwen3-Omni waveform runtime

1.1 Establish the native Thinker baseline

- Build the current Qwen3-Omni Q4_K_M GGUF path in `llama.cpp`.
- Validate audio input through the existing multimodal projector path before adding output synthesis.
- Record a reproducible baseline for:
 - tensor inventory, names, dimensions, data types, and GGUF metadata;
 - loaded-weight memory by module and backend;
 - CUDA graph workspace and allocator overhead;
 - Thinker KV allocation at each supported context size;
 - peak VRAM during prefill and decode;
 - prefill latency, decode latency, and throughput.
- Separate decimal file size, binary device allocation, and measured CUDA peak memory. Do not infer deployability from GGUF file size alone.
- Preserve fixed reference inputs and Thinker logits for regression testing throughout the port.

1.2 Build an audio-only GGUF package

- Retain the Q8 AuT encoder and every audio projection, normalization, and temporal-encoding tensor required by the Thinker and Talker.
- Remove the vision encoder, vision-only projector tensors, and vision-only runtime paths.
- Preserve sensitive audio projection and normalization tensors at their selected higher precision where conversion requires it.
- Validate the audio-only package against the original multimodal package on identical audio inputs: final AuT and projector representations, Thinker logits, internal generated token sequence, memory, and latency.

1.3 Extend GGUF conversion for speech output

- Convert and package Q8_0 Talker, BF16 MTP, BF16 Code2Wav, speaker/voice-conditioning embeddings, and all metadata required for streaming generation.
- Define independent, versioned tensor namespaces for Thinker, AuT, Talker, MTP, and Code2Wav.
- Verify tensor names, dimensions, strides, data types, checksums, and correspondence with the reference PyTorch weights.
- Keep norms, routers, embeddings, and sensitive output heads at BF16 where quantization offers little benefit or disproportionate risk.

1.4 Port the Talker to GGML

- Implement the 20-layer Talker MoE graph and its independent KV state.
- Implement prompt processing and transfer of model-native conditioning representations from the Thinker.
- Preserve Q8 Talker weights exactly; CPU residency changes placement, not numerical precision.
- Validate dense sublayers, router logits and expert identifiers, expert outputs, layer hidden states, Talker logits, and primary codec predictions against fixed PyTorch references.

1.5 Port the hierarchical MTP loop

- Generate the primary codec value from the Talker.
- Autoregressively generate the remaining residual codebooks through MTP, conditioned on preceding codebooks in the same codec frame.
- Maintain fixed, bounded MTP state for the current codec frame.
- Validate every codebook prediction and intermediate MTP hidden state against the reference implementation.
- Reuse one MTP scratch allocation across residual-codebook substeps.

1.6 Port streaming Code2Wav

- Reconstruct the complete causal Code2Wav graph with GGML CPU operators, adding kernels only where profiling demonstrates a gap.
- Maintain bounded causal left context and produce incremental 24 kHz PCM.
- Resample only at the external audio interface when another sample rate is required.
- Validate waveform reconstruction from fixed codec sequences before integrating generation.
- Measure CPU real-time factor, thread count, memory bandwidth, and interference with pinned expert storage.

1.7 Implement the Thinker–Talker scheduler

- Coordinate incoming chunks, AuT encoding, Thinker prefill/generation, Talker conditioning, MTP codec generation, and Code2Wav streaming.
- Preserve separate state domains for AuT frames, Thinker positions, Talker positions, codec frames, and PCM samples.
- Implement asynchronous Thinker–Talker operation and bounded inter-module queues.
- Support cancellation, cleanup, and phase transitions during barge-in.
- Keep the entire serving path in one native C++ process.

2 Static L4 memory architecture

2.1 Apply the fixed precision policy

- Thinker: Q4_K_M.
- AuT/projector: Q8_0, with sensitive tensors at higher precision where required.
- Talker: Q8_0.
- MTP: BF16.
- Code2Wav: BF16 on CPU.
- Thinker and Talker KV: Q8_0.
- Norms, routers, embeddings, and sensitive heads: BF16 where required.
- Vision tensors: absent.

2.2 Implement phase-overlaid module residency

- Keep the Thinker resident throughout the session.
- During ordinary listening, place AuT in the shared modality arena.
- During VoxZip-reference listening, allow AuT and Whisper to coexist only if measured allocation remains safe.
- During sequential VoxZip execution, release AuT before loading Whisper into the same arena.
- During speaking, release AuT and Whisper and use the arena for the Talker expert-slot pool.
- Keep Code2Wav on CPU throughout output synthesis.
- Maintain CPU-side endpointing and interruption detection while speaking.

Let M_{AuT} , M_{Whisper} , and $M_{\text{TalkerPool}}$ denote each modality module’s weights and private workspace, and let M_{overlap} contain concurrently live staging, non-reusable scratch, and allocator overhead. The shared-arena live set is phase-specific:

$$M_{\text{shared}}(p) = \begin{cases} M_{\text{AuT}}, & p = \text{ordinary listening,} \\ M_{\text{AuT}} + M_{\text{Whisper}} + M_{\text{overlap}}, & p = \text{concurrent VoxZip listening,} \\ \max(M_{\text{AuT}}, M_{\text{Whisper}}), & p = \text{sequential VoxZip endpointing,} \\ M_{\text{TalkerPool}}, & p = \text{speaking.} \end{cases}$$

The total phase peak also includes the always-resident Thinker, KV, transfer staging, runtime allocation, fragmentation, and reserve. Measured phase totals, rather than this accounting identity alone, determine L4 acceptance.

2.3 Build a lifetime-managed GGML workspace

- Replace separate maximum-sized workspaces with a lifetime-aware shared arena.
- Reuse temporary allocations across AuT, Whisper when enabled, Thinker, Talker, repeated MTP substeps, and Code2Wav chunks.
- Recycle logits, temporary embeddings, and emitted codec/PCM buffers after their last consumer.
- Tune batch and microbatch sizes for one streaming conversation rather than generic serving.
- Instrument high-water marks for weights, KV, scratch, transfer staging, and allocator overhead separately.

3 Exact Q8 Talker expert residency

3.1 Divide permanent and cacheable Talker weights

- Keep dense attention tensors, routers, embeddings, norms, and codec output heads resident during output generation.

- Keep authoritative Q8 copies of all routed experts in pinned CPU RAM.
- Allocate an initial fixed 1.5 GiB GPU expert-slot pool, then tune it from measured hit-rate and peak-memory curves.
- Never approximate, prune, or substitute a routed expert; a miss changes latency, not weights.

3.2 Implement expert-slot addressing

- Maintain an expert-to-slot mapping independently for each Talker layer.
- Dispatch cache hits directly to resident Q8 tensors.
- On a miss, select a victim, transfer the exact expert asynchronously into a staging slot, wait only for uncovered transfer time, atomically update the mapping, and execute it.
- Track raw transfer time and uncovered stall time separately:

$$T_{\text{stall}} = \max(0, T_{\text{copy}} - T_{\text{overlap}}).$$

3.3 Implement deterministic latency controls

- Use pinned host storage, persistent GPU slots, a dedicated DMA stream, and per-layer active/staging buffers.
- Prefetch experts from current generation state and trace-derived hot sets.
- Begin with fixed-frequency hot sets, LRU, and segmented LRU.
- Collect L4 routing traces and replay them offline to calculate reuse distance, working sets, hit curves, and oracle performance.
- Add predicted future-stall eviction only if deterministic policies leave a meaningful measured gap to the oracle.

3.4 Isolate playback from inference jitter

- Place an initial 320 ms PCM buffer between Code2Wav and the audio interface.
- Track occupancy as

$$B_{t+1} = B_t + T_{\text{generated}} - T_{\text{elapsed}}.$$

- Prefetch the initial Talker hot set before playback begins.
- Keep occupancy above a measured threshold and use bounded clause-boundary pause extension only as an underflow safeguard.
- Measure audible interruptions, underflows, first-audio latency, and 99th-percentile generation real-time factor.

4 Native bounded-KV foundation

4.1 Implement native KV instrumentation

- Begin only after short-context waveform inference is correct.
- Track Thinker positions by conversational block and associate audio positions with original AuT intervals.
- Record per-layer/head KV allocation, attention statistics, boundaries, and eviction decisions without Python serving.
- Distinguish weights, KV, workspace, transfer staging, and allocator overhead.
- Maintain a byte-bounded in-memory CPU ledger containing only records required to reconstruct the protected, historical, and exact-recent regions inside the active context budget.
- Keep raw audio in a fixed-duration ring buffer. Serialize completed audit records to an append-only per-run disk journal, then release each record and source interval after its bounded live consumers finish.

4.2 Define the fixed Thinker budget

- Begin with 4,096 Q8 KV positions: 512 protected positions, 2,560 exact recent positions, and 1,024 historical positions.
- Make the recent/historical boundary elastic while keeping the total fixed.
- Preserve the protected prefix, current input, and immediately preceding conversational neighborhood.
- Exclude confidently detected silence, ringback, and hold music before inference.
- Preserve wall-clock time and non-neural runtime state outside KV.

4.3 Implement bounded context reconstruction

- Keep recent blocks as exact native representations.
- Retain compacted candidates only for blocks addressable by the active protected, historical, and exact-recent budgets. Persist older audit evidence to disk without retaining it in process memory.
- At budget pressure, rebuild at a safe boundary from

[protected prefix; compacted history; exact recent context].

- Re-prefill this bounded sequence to produce ordinary model KV.
- Apply consistent history decisions to Talker conditioning state that spans turns; keep MTP and Code2Wav independently bounded.
- Measure rebuild frequency, amortized prefill cost, and divergence from full context.
- Enforce hard byte limits for the live ledger, raw-audio ring, compacted-candidate store, and inter-module queues; sample their high-water marks and process RSS during soak tests.

5 Staged bounded-context experiments

5.1 Stage A: full AuT with a recent window

- Retain the protected prefix and newest complete blocks that fit.
- Evict the oldest unprotected complete block.
- Use no compression, learned representation, or attention selection.
- Establish the simplest fixed-budget quality, latency, and memory curve.

5.2 Stage B: fixed-rate AuT temporal pooling

- Insert the hook after final AuT projection and before Thinker prefill.
- For $E = [e_1, \dots, e_L]$ and factor c , compute

$$r = \left\lceil \frac{L}{c} \right\rceil, \quad z_j = \frac{1}{|\mathcal{G}_j|} \sum_{k \in \mathcal{G}_j} e_k.$$

- Test $c \in \{2, 4, 8\}$, approximately one vector per 160, 320, and 640 ms at AuT’s 12.5 Hz rate.
- Never pool across speech boundaries, speaker turns, independent events, or streaming discontinuities.
- Compare dense renumbering, first-frame time, and center-frame time for temporal positions.
- Stream with at most $c - 1$ unfinished frames and flush residual groups at boundaries.
- Call this fixed-rate native AuT temporal pooling, not semantic anchoring.
- Implement it directly in GGML/C++; it has no new weights to serialize.

5.3 Stage C: reproduce VoxZip semantic anchoring

- Reproduce the published first stage on the A100 with F16 Whisper-Turbo.
- Attempt the published backbone precision and context length only if the available single A100 can hold them. Otherwise run the same method on the target Q4 backbone and explicitly report the precision/context deviation from the published eight-A100 setup.
- Whisper supplies transcript text, timestamps, and confidence; Qwen supplies AuT representations and its text embedding matrix.
- Retain Whisper segments above the published confidence threshold of 0.3 and map each interval to its AuT range.
- For L_t Qwen transcript tokens, partition L_a AuT vectors into L_t groups and compute

$$\tilde{e}_{a,j} = \frac{1}{|\mathcal{G}_j|} \sum_{k \in \mathcal{G}_j} e_{a,k}.$$

- Fuse pooled audio and corresponding Qwen text embedding:

$$e_{f,j} = \tilde{e}_{a,j} + e_{t,j}.$$

- Preserve background intervals without high-confidence transcripts and concatenate all intervals chronologically.
- Use recent-window retention so Stage C isolates semantic anchoring from eviction.
- Verify the A100 reproduction before adapting it to the L4.

5.4 Native L4 Whisper adaptation for Stages C and D

- Integrate `whisper.cpp` into the native process; no Python server is permitted.
- Attempt Q8_0 before Q5_0. Compare both with F16 on transcripts, timestamps, anchor count, fused-anchor similarity, and downstream output.
- **Concurrent path:**
 1. keep Q8 Whisper resident while listening if peak memory permits;
 2. process bounded chunks on a separate, lower-priority CUDA stream;
 3. prioritize AuT and Thinker work;
 4. finalize Whisper immediately after endpointing;
 5. construct anchors;
 6. release AuT and Whisper before loading Talker expert slots.
- Under concurrency, added latency is approximately

$$T_{\text{added}} = T_{\text{Whisper tail}} + T_{\text{anchor assembly}}.$$

- **Sequential path:**
 1. AuT processes the complete turn and retains projected representations;
 2. endpoint detection closes the turn;
 3. release AuT weights and workspace;
 4. load Q8 Whisper into the shared arena;
 5. transcribe the buffered turn and generate timestamps;
 6. construct VoxZip anchors;
 7. release Whisper;
 8. prefill Thinker and transition to Talker generation.

- Sequential endpointing reuses the arena, so its modality contribution is $\max(M_{\text{AuT}}, M_{\text{Whisper}})$. Concurrent listening instead adds AuT, Whisper, and overlap allocations. Speaking is a separate phase whose modality contribution is $M_{\text{TalkerPool}}$.
- Keep weights in pinned CPU RAM, pre-create reusable state, and use asynchronous DMA to amortize transfer and initialization.
- If Q8 fails both residency paths, repeat with Q5_0. Q5 is a fallback, not reference precision.
- Also benchmark CPU-only Whisper; retain it only if its real-time and post-endpoint latency gates pass without interference.

5.5 Stage D: complete VoxZip reproduction

The method-faithful contract first implements VoxZip Section 3.2.2 without substituting local retention rules.

- Add temporally decayed accumulated-attention eviction to Stage C anchors.
- Reproduce published decay $\gamma = 0.95$ before tuning.
- Reproduce the policy independently per layer. For a budget of B positions, retain the first $T = 4$ attention-sink positions, assign $M = \lfloor 0.20B \rfloor$ positions to the recent window, and assign the remaining $N = B - T - M$ positions to historical entries with the largest decayed accumulated-attention scores.
- Update scores before selection using

$$A^{(t)} = \gamma[A^{(t-1)}, 0] + \text{softmax}\left(\frac{q_t K_t^\top}{\sqrt{d}}\right),$$

where $q_t \in \mathbb{R}^{1 \times d}$ is the current query, $K_t \in \mathbb{R}^{t \times d}$ is the current layer’s key cache, d is the head dimension, and $[A^{(t-1)}, 0]$ appends the new position with zero prior score.

- Run the published $T/M/N$ policy before a project-specific protected-prefix adaptation. Label any run that replaces the published allocation as **VoxZip-local-policy**.
- Reproduce the method on the A100 first, documenting any single-GPU deviation, then validate Q8 and, if required, Q5 L4 adaptations.
- Treat D as the strongest external-ASR reference, not the intended final single-model method.

5.6 Stage E: fixed pooling with the same temporal eviction

- Apply the exact retention budget and temporal policy from the corresponding D run: published D with published E, and **VoxZip-local-policy** D with the same local-policy E.
- Match initial representation length to VoxZip anchor count or enforce identical total KV bytes.
- Hold backbone precision and runtime fixed so the D–E difference isolates Whisper-derived semantic anchoring.
- Establish the strongest train-free, ASR-free native baseline.

5.7 Stage F: learned monotonic AuT compactor

- Implement one selected learned monotonic architecture, not separate learned and monotonic variants.
- For length L and compression c , emit $r = \lceil L/c \rceil$ chronologically ordered vectors.
- Associate query q_j with temporal neighborhood $\mathcal{N}_j$:

$$u_j = \text{Attention}(q_j, E[\mathcal{N}_j]), \quad z_j = \bar{e}_j + \text{MLP}(u_j).$$

- Initialize near fixed pooling, preserving ordered outputs and unambiguous representative timestamps.
- Freeze AuT, Thinker, Talker, MTP, and Code2Wav; train only the compactor initially.

- Distill continuation-token behavior from full-input A100 teacher runs:

$$\mathcal{L}_{\text{KL}} = \sum_t D_{\text{KL}}(p_{\text{full}}(y_t) \| p_{\text{compact}}(y_t)).$$

- Optionally match selected hidden states at common continuation-token positions:

$$\mathcal{L}_h = \sum_{\ell \in \mathcal{S}} \sum_{t \in \text{continuation}} \|h_{\ell,t}^{\text{full}} - h_{\ell,t}^{\text{compact}}\|_2^2.$$

- Train with $\mathcal{L} = \mathcal{L}_{\text{KL}} + \lambda_h \mathcal{L}_h$.
- Use checkpointing, minimal microbatches, and cached teacher outputs as required by A100 memory.
- Compare fixed and learned methods at identical output lengths and temporal policies.
- Advance only if the paired improvement at the same budget clears the material-improvement threshold frozen by the Phase 0 evaluation contract.

5.8 Export and execute the learned compactor natively

- Define GGUF metadata for architecture, widths, compression rate, query count, temporal neighborhood, and precision.
- Convert compactor parameters into GGUF tensors traceable by name and checksum to the PyTorch checkpoint.
- Implement the GGML graph at the validated post-projection/pre-Thinker interface.
- Validate embeddings and Thinker logits against PyTorch.
- Measure weight allocation, workspace, latency, and L4 peak memory.
- This export is compactor-specific; all other KV policies still require native C++/GGML execution even without new weights.

5.9 Stage G: learned compactor with temporal eviction

- Combine F’s learned native anchors with the identical decay policy and byte budget used in D and E.
- Compare G versus F for eviction value, G versus E for learned versus fixed transcript-free compaction, and G versus D for native versus transcript-derived anchors.
- Adopt G only if it improves on E without violating L4 memory or latency gates.

6 Parallel execution and integration contracts

6.1 Freeze shared contracts before parallel implementation

- Pin exact commits for `llama.cpp`, `whisper.cpp`, the Qwen reference implementation, model repositories, CUDA, compiler, and GGUF conversion scripts.
- Publish one canonical tensor manifest for every module: source name, GGUF name, shape, stride, data type, quantization, checksum, and intended backend.
- Define typed C++ interfaces for AuT output, Thinker conditioning, Talker output, MTP codec frames, Code2Wav chunks, expert-cache requests, and bounded-context rebuilds before agents implement their internals.
- Make ownership, lifetime, device, CUDA stream, synchronization event, mutability, and alignment explicit for every buffer crossing a module boundary.
- Use distinct types for audio samples, AuT frames, Thinker positions, Talker positions, codec frames, and wall-clock time; prohibit implicit conversion between these coordinate systems.
- Version GGUF metadata and the native module ABI. Reject incompatible artifacts at load time with a diagnostic rather than continuing with inferred defaults.

- Freeze a Phase 0 evaluation and deployment contract containing the corpus manifest, paired comparison keys, correctness tolerances, equivalence/non-inferiority margins, material-improvement thresholds, latency/underflow gates, workload, repetition rules, and certified L4 host profile.

6.2 Create golden fixtures and machine-readable outputs

- Generate small, deterministic reference fixtures from the pinned PyTorch implementation for every module and cross-module boundary.
- Store inputs, tensor outputs, shapes, data types, checksums, sampling parameters, seeds, and numerical tolerances together in a versioned fixture manifest.
- Include fixtures for variable audio-chunk sizes, a final partial chunk, empty/background intervals, long turns, interruption, context rebuild, and every supported codec-frame boundary.
- Record the first divergent module, layer, tensor, and position so integration failures can be localized rather than diagnosed only from the final waveform.
- Emit benchmark and correctness results as versioned JSON records in addition to human-readable logs.

6.3 Partition work into dependency-safe work packages

- **Foundation:** pinned environment, tensor manifests, fixture-contract design, measurement-contract design, evaluation/deployment acceptance contract, and reference-runner design may begin together. Fixture generation and manifest-derived memory/cache spikes follow their validated inputs; the ABI freezes after those outputs reconcile.
- **Parallel native ports:** audio-only AuT packaging, Talker graph, MTP loop, Code2Wav graph, and GGUF conversion may proceed independently after the foundation contracts freeze.
- **Scheduler integration:** begins after each module passes its boundary fixture; it owns queues, phase transitions, cancellation, and end-to-end state.
- **L4 residency:** shared allocator and static memory tracing may proceed with synthetic module loads; expert-cache execution begins after Talker routing and expert-call traces are stable.
- **Bounded context:** instrumentation and Stage A may proceed once the Thinker input/KV interface is stable. Stages B/E, C/D, and F/G then form three parallel experimental branches sharing the same byte-budget and evaluation harness.
- **Integration owner:** one workstream owns ABI changes, fixture-version changes, merge ordering, and release-candidate assembly. Individual agents do not silently change shared contracts.

6.4 Enforce merge and release discipline

- Keep each port and policy behind compile-time or runtime feature flags until its fixtures pass.
- Require every merge to declare changed tensors, interfaces, allocations, numerical tolerances, and benchmark deltas.
- Run CPU smoke tests on every change and CUDA correctness/performance tests on scheduled integration commits.
- Maintain one end-to-end manifest containing binary hashes, GGUF hashes, command line, hardware, driver, runtime settings, and evaluation-corpus version.
- Do not optimize a numerically divergent module. Correctness gates precede performance gates, which precede policy comparisons.

7 Integrated system evaluation

The Phase 0 evaluation contract assigns every use of “equivalent,” “material improvement,” “meets latency,” and “passes fidelity” to a named metric, paired comparison, margin, confidence-interval rule, corpus slice, and workload.

7.1 Evaluation corpus and protocol

- Freeze a versioned corpus spanning clean speech, 8 kHz telephony speech, background noise, music, silence, multiple speakers, interruptions, and long multi-turn interactions.
- Stratify results by duration, channel condition, language, speaker count, and interaction pattern rather than reporting only a global mean.
- Include controlled long-context probes for entity/value retention, delayed reference, speaker attribution, instruction persistence, and information introduced before and after context rebuilds.
- Use identical prompts, audio, sampling parameters, seeds where applicable, and output limits across compared systems.
- Separate warm and cold runs; declare warm-up count, repetitions, CPU affinity, GPU clocks/power mode, CUDA synchronization points, and measurement interval.
- Report paired differences and confidence intervals.
- Freeze equivalence margins, non-inferiority margins, material improvement thresholds, and deployment gates during Phase 0 before selecting the final method.
- External ASR or quality models may score evaluation outputs offline, but they are never part of the deployed inference path.

7.2 Port correctness

- Validate module-level equivalence for AuT, Thinker, Talker dense layers and routers, routed experts, MTP, and Code2Wav against the pinned reference implementation.
- Compare intermediate tensors at agreed checkpoints using absolute error, relative error, cosine similarity, top- k agreement, and first-divergence location as appropriate.
- Compare Thinker and Talker token distributions, selected experts, primary codec values, all residual codebooks, and final PCM.
- Report exact codec-token agreement and waveform error separately; floating-point waveform differences must not conceal discrete-code divergence.
- Verify streaming equivalence across chunk sizes and explicitly test the final incomplete audio chunk so no tail audio is silently discarded.
- Exercise cancellation, barge-in, repeated listen/speak transitions, context reconstruction, allocator reuse, and clean teardown under sanitizers and long-running stress tests.

7.3 L4 feasibility and resource accounting

- Measure peak and steady-state VRAM separately for ordinary listening, concurrent VoxZip listening, sequential VoxZip endpointing, Thinker prefill, Talker startup, sustained synthesis, barge-in, and context rebuild.
- Account separately for weights, KV, graph workspace, the shared arena, expert slots, and transfer staging.
- Record CUDA runtime allocation, fragmentation, and unclaimed reserve separately.
- Report host RAM, pinned RAM, CPU memory bandwidth, PCIe bytes and transactions, and Code2Wav interference with expert transfers.
- Bind deployment claims to the certified host CPU, RAM, pinned-memory limit, PCIe topology/link state, operating system, driver, CUDA version, power mode, and thermal controls. Treat unmatched hosts as exploratory unless the deployment contract accepts them.
- Compare concurrent and sequential Whisper residency, including load time, reusable-state initialization, endpoint tail, and shared-arena high-water mark.
- Measure cold start, warm start, endpoint-to-first-audio latency, p50/p95/p99 real-time factor, and sustained throughput.
- Run repeated phase-transition and long-duration soak tests to detect leaks, allocator growth, fragmentation, race conditions, and delayed OOM failures.

7.4 Q8 Talker expert-residency evaluation

- Use an all-resident Q8 Talker on suitable reference hardware as the fidelity oracle. Verify identical authoritative Q8 tensor bytes by checksum, then compare routers, selected experts, logits, codec codes, and waveform against the paired equivalence margins.
- Measure hit rate, reuse distance, per-layer occupancy, miss bursts, prefetch precision/recall, wasted transfer bytes, raw copy time, uncovered stall time, and stall contribution to PCM-buffer depletion.
- Sweep expert-slot capacity and compare naïve CPU execution, static hot sets, LRU, segmented LRU, and a trace-replay oracle under identical routing traces.
- Evaluate routing-distribution shifts across speakers, voices, languages, channel conditions, and conversational regimes; do not tune only on the trace used to choose the hot set.
- Report PCM-buffer occupancy distributions, underflow rate, audible-interruption rate, and latency added by recovery from clustered cache misses.

7.5 Whisper reference and deployment adaptation

- Compare F16 A100, Q8.0 L4, and Q5.0 L4 Whisper-Turbo on identical buffered turns.
- Measure word error, insertions/deletions, timestamp error, confidence-threshold crossings, transcript-token-count disagreement, anchor-count disagreement, fused-anchor similarity, and downstream output divergence.
- Compare concurrent GPU, sequential GPU, and CPU-only execution in accuracy, peak memory, endpoint tail, interference with AuT/Thinker work, and total first-audio latency.
- Prefer Q8 unless the measured envelope fails. Admit Q5 only within a predeclared downstream tolerance of F16 and only after sequential Q8 is rejected.

7.6 Bounded-context quality

- Compare full history and Stages A–G at identical total KV bytes, with protected-prefix bytes reported separately.
- Report representation length, compression ratio, KV allocation, rebuild frequency, rebuild latency, retained-history horizon, and divergence from the full-history teacher.
- Measure semantic task quality, exact entity/value retention, delayed-reference accuracy, speaker attribution, instruction persistence, and consistency across context rebuilds.
- Evaluate duration curves rather than a single endpoint: quality versus interaction length and versus retained KV bytes.
- For temporal eviction, report retained-position distributions, score aging, churn, and sensitivity to γ and budget. For compactors, report output-length calibration and temporal coverage.
- Separate representation effects from eviction effects using the paired comparisons A/B, C/D, B/E, F/G, D/E, and E/G.

7.7 Output fidelity and interaction quality

- Evaluate semantic equivalence of the generated response independently from acoustic rendering quality.
- Report intelligibility, pronunciation of rare words and numbers, speaker similarity, spectral distortion, prosody, and human preference where feasible.

- Measure endpointing delay, first-audio latency, interruption response, false interruption, and turn overlap; also report uninterrupted output and recovery after barge-in.
- Report failures by causal layer: input recognition, Thinker decision, context policy, Talker codec generation, Code2Wav rendering, expert stall, or scheduler fault.

7.8 Required ablations

- Thinker/Talker F16 KV versus Q8 KV on short contexts, isolating KV quantization from bounded-context policy.
- Q8 Talker all-resident versus exact CPU-backed residency; Q4 Talker is an optional memory-fidelity stress point, not the deployment baseline.
- Naïve CPU expert execution versus persistent slots; static hot set versus LRU versus segmented LRU versus oracle.
- Independent modality residency versus phase overlay; concurrent versus sequential Whisper; GPU versus CPU Code2Wav where feasible.
- Stage A recent window versus Stage B fixed pooling.
- Stage C versus Stage D holds semantic anchoring fixed and adds temporal eviction. Stage B versus Stage E holds fixed pooling constant and adds temporal eviction. Stage D versus Stage E holds eviction, KV bytes, backbone precision, and runtime fixed to isolate transcript-derived semantic anchoring.
- Stage B versus Stage F at identical compressed length; Stage E versus Stage G at identical KV bytes.
- Fixed-pooling factors $c \in \{2, 4, 8\}$, temporal-position conventions, compactor output depth, and temporal-decay sensitivity.

7.9 Acceptance and sequencing gates

1. Every native module passes its golden fixtures before scheduler integration.
2. Correct short-context native waveform inference precedes bounded-context work.
3. The static runtime fits without relying on history compression for startup survival.
4. Measured peak remains below 22 GiB under worst-case tested phases, leaving operational reserve beneath the hard 24 GB ceiling.
5. CPU-backed and all-resident execution load identical authoritative Q8 tensor bytes, and paired output differences remain inside the Phase 0 fidelity-equivalence margins while satisfying its latency/underflow gates.
6. Stage A establishes the fixed-budget native baseline. Stages C and D reproduce VoxZip before its components are replaced.
7. Stage E becomes the train-free ASR-free baseline. Stage F advances to GGUF only if its paired improvement over B at matched length clears the Phase 0 material-improvement threshold.
8. Stage G becomes the deployment candidate only if its paired improvement over E at matched bytes clears that threshold and passes all L4 latency, memory, stability, and output-fidelity gates.
9. The release candidate completes long-duration soak, repeated barge-in, and allocator-reuse tests with no OOM, leak, deadlock, corrupted state, or unbounded latency growth.

Primary references

- Qwen Team, Qwen3-Omni Technical Report.
- Qwen, Qwen3-Omni-30B-A3B-Instruct model and configuration.
- ggml-org, Qwen3-Omni GGUF artifacts.
- VoxZip, Semantic-Anchored Temporal KV Cache Compression for Long-Context Audio Inference.
- ggml-org, whisper.cpp native runtime, CUDA backend, quantization, and model inventory.
- ggml-org, Qwen3-Omni final-partial-audio-chunk regression and fix history.
- NVIDIA, L4 specifications.